INFORME TÉCNICO: SIMULACIÓN DE ATAQUES EN UN PROYECTO DE SOFTWARE

Juan Jose Botero Muriel

Prueba y caidad de Software
Ingeniería de Software
Helio Ruber López Carreño

Corporación Tecnológica del Oriente
Noviembre, 2026

1. Introducción y Contexto Técnico

La automatización de pruebas de software se ha consolidado como una práctica fundamental en el desarrollo ágil y la integración continua (CI/CD). Su objetivo principal es acelerar la validación funcional, garantizar la repetibilidad de las pruebas y obtener una retroalimentación inmediata ante cambios en el código. El presente taller individual documenta el diseño, desarrollo, ejecución y verificación de un conjunto de scripts automatizados aplicados a dos capas críticas de un sistema: la interfaz de usuario (Formulario Web) y la capa de servicios (API REST).

2. Arquitectura y Estructura del Proyecto

Siguiendo las orientaciones impartidas por el docente Helio Ruber López Carreño en la sesión sincrónica, se estructuró un proyecto modular en Python con separación clara de responsabilidades, generación autónoma de carpetas de evidencia y reporte web consolidado.

Archivo / Carpeta	Tipo	Descripción y Responsabilidad
test_formulario.py	Script Python	Automatización de UI con Selenium WebDriver. Rellena campos, valida 'Teléfono' y envía el formulario.
test_rest_api.py	Script Python	Automatización de API con Requests. Ejecuta peticiones GET y POST validando payloads y códigos HTTP.
ejecutar_pruebas.py	Script Principal	Orquestador principal. Ejecuta los tests, recopila métricas y compila el reporte HTML.
evidencias/	Directorio PNG	Generado automáticamente. Almacena las capturas de pantalla de cada paso ejecutado.
reportes/	Directorio HTML	Generado automáticamente. Contiene 'reporte_pruebas.html' con la matriz final de resultados.

3. Desarrollo Técnico y Modificaciones Realizadas

3.1 Script de Formulario Web (test_formulario.py)

Se automatizó la navegación sobre un formulario web utilizando **Selenium WebDriver** y **webdriver- manager**. Como requisito clave del docente, el script fue personalizado agregando la interacción con un nuevo campo: **Teléfono (custtel)**, el cual no venía en la estructura base.

Flujo automatizado:

1. Apertura del navegador Chrome y carga de la URL objetivo.
2. Diligenciamiento de Nombre ('custname'), Email ('custemail') y el campo personalizado **Teléfono ('custtel')**.
3. Selección dinámica de opciones de radio button (Tamaño Grande).
4. Envío del formulario ('submit') y captura de confirmación del servidor.

3.2 Script de Servicio REST (test_rest_api.py)

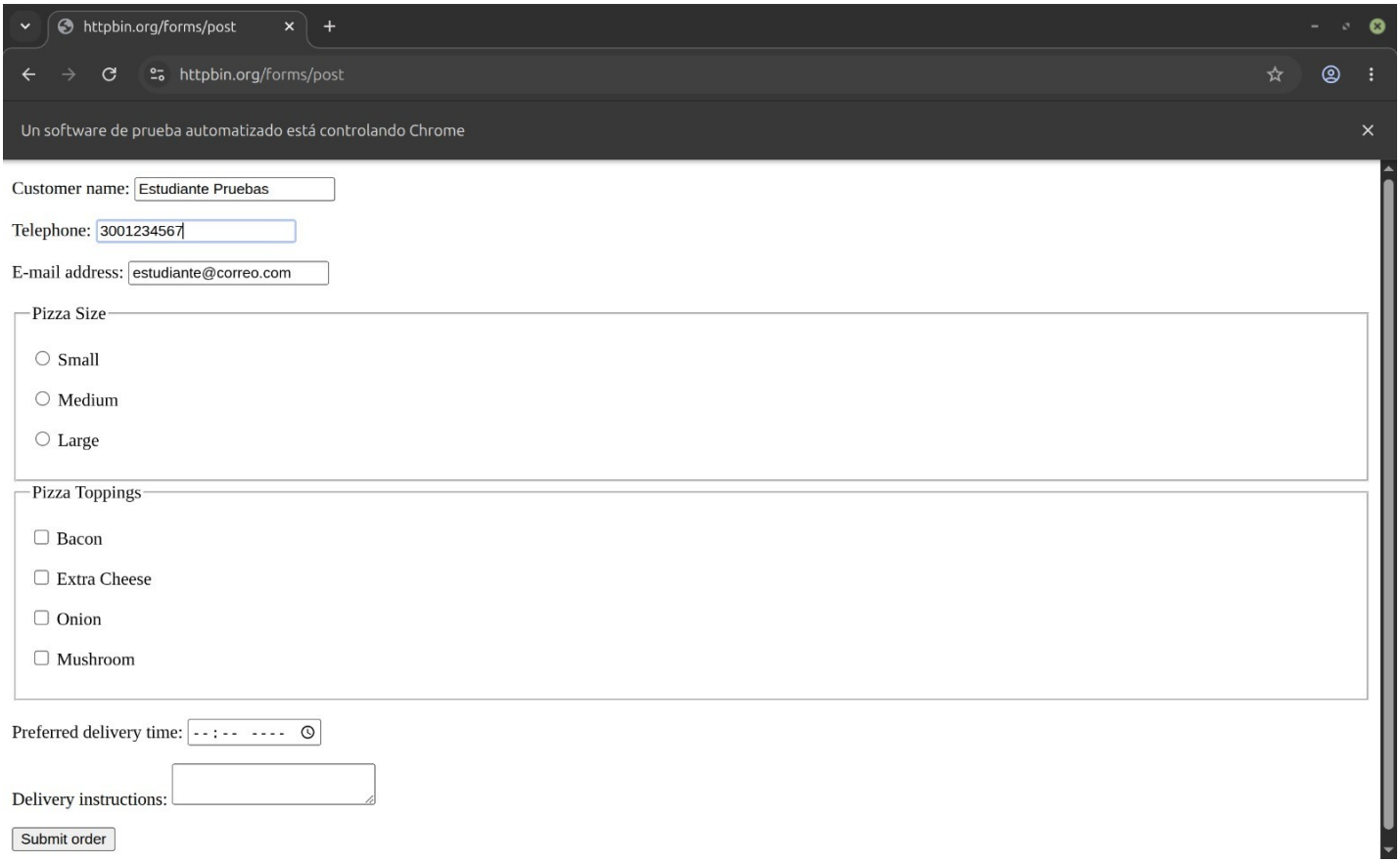
Para la capa de servicios, se utilizó la librería **Requests** para enviar peticiones HTTP a endpoints REST.

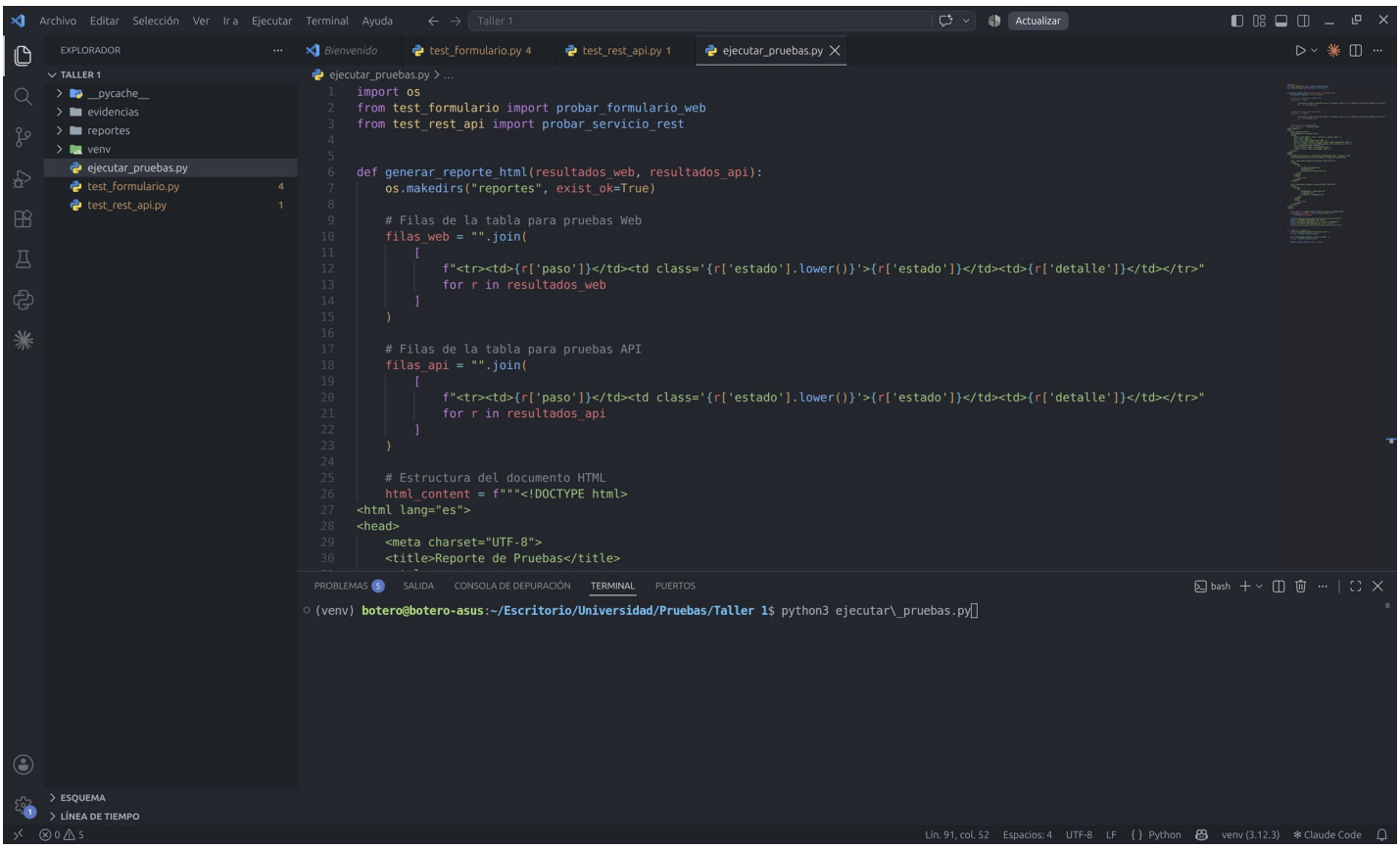
- **Prueba GET:** Consume '/posts/1' y verifica que el servidor retorne HTTP 200 OK y la estructura de datos.
- **Prueba POST (Personalizada): Envía un objeto JSON agregando el parámetro adicional 'categoria_prueba': 'AA1_Calidad_Software', verificando la creación exitosa del recurso con código HTTP 201 Created.**

4. Evidencias de Ejecución y Reporte Consolidado

La ejecución del script principal ejecutar_pruebas.py finalizó de forma exitosa, procesando la totalidad de los casos de prueba sin fallos. A continuación se presenta el consolidado de la matriz de resultados generada automáticamente en el archivo HTML:

Capa Probada	Paso / Operación de Prueba	Estado	Detalle de Validación / Respuesta
Formulario Web	Abrir Formulario Web	**PASÓ**	Página cargada correctamente en navegador Chrome
Formulario Web	Llenar Campos (Nombre, Email, Teléfono)	**PASÓ**	Campo adicional 'Teléfono' validado y diligenciado
Formulario Web	Seleccionar Opción	**PASÓ**	Opción 'Tamaño Grande' seleccionada dinámicamente
Formulario Web	Enviar Formulario	**PASÓ**	Formulario procesado con respuesta de servidor HTTP 200 OK
Servicio REST	GET /posts/1	**PASÓ**	Status 200 OK - Registro obtenido correctamente
Servicio REST	POST /posts (Crear Registro)	**PASÓ**	Status 201 Created - Objeto creado con campo 'categoria_prueba'


httpbin.org/forms/post
httpbin.org/forms/post
Un software de prueba automatizado está controlando Chrome
Customer name: Estudiante Pruebas
Telephone: 3001234567
E-mail address: estudiante@correo.com
Pizza Size
Small
Medium
Large
Pizza Toppings
Bacon
Extra Cheese
Onion
Mushroom
Preferred delivery time: --:-- ----
Delivery instructions:
Submit order


Archivo Editar Selección Ver Ir a Ejecutar Terminal Ayuda
Taller 1
Actualizar
EXPLORADOR
TALLER 1
__pycache__
evidencias
reportes
venv
ejecutar_pruebas.py
test_formulario.py 4
test_rest_api.py 1
Bienvenido
test_formulario.py 4
test_rest_api.py 1
ejecutar_pruebas.py
ejecutar_pruebas.py > ...
import os
from test_formulario import probar_formulario_web
from test_rest_api import probar_servicio_rest
def generar_reporte_html(resultados_web, resultados_api):
os.makedirs("reportes", exist_ok=True)
Filas de la tabla para pruebas Web
filas_web = "".join(
[
f"<tr><td>{r['paso']}</td><td class='{r['estado'].lower()}'>{r['estado']}</td><td>{r['detalle']}</td></tr>"
for r in resultados_web
]
)
Filas de la tabla para pruebas API
filas_api = "".join(
[
f"<tr><td>{r['paso']}</td><td class='{r['estado'].lower()}'>{r['estado']}</td><td>{r['detalle']}</td></tr>"
for r in resultados_api
]
)
Estructura del documento HTML
html_content = f"""<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Reporte de Pruebas</title>
PROBLEMAS 5
SALIDA
CONSOLA DE DEPURACIÓN
TERMINAL
PUERTOS
bash
(venv) botero@botero-asus:~/Escritorio/Universidad/Pruebas/Taller 1$ python3 ejecutar_pruebas.py
ESQUEMA
LÍNEA DE TIEMPO
Lín. 91, col. 52
Espacios: 4
UTF-8
LF
Python
venv (3.12.3)
Claude Code